Sprawozdanie 3

Metody Monte Carlo

Tadeusz Jagniewski (277522), Paweł Nowak (282223)

04 May 2026

Spis treści

Zadanie 1: Algorytm Metropolisa-Hastingsa (5 pkt) . . . 1
Rozwiązanie . . . 2
Zadanie 2: Próbnik Gibbsa (3 pkt) . . . 11
Rozwiązanie . . . 11
Zadanie 3: Estymacja mocy testów (4 pkt) . . . 20
Rozwiązanie . . . 20
Zadanie 4: Optymalizacja Monte-Carlo (2 pkt) . . . 25
Rozwiązanie . . . 25

Zadanie 1: Algorytm Metropolisa-Hastingsa (5 pkt)

Chcemy estymować wartość całki

$$I = \int_{-\infty}^{\infty} \frac{\sqrt{|x|}\cos x}{(1+2x^2)^{5/3}} \mathrm{d}x$$

1. Napisz program generujący łańcuch Markowa $(X_j)_{j\in\mathbb{N}_0}$ przy pomocy algorytmu Metropolisa-Hastingsa o rozkładzie stacjonarnym o gęstości

$$f(x) = M_f \frac{\sqrt{|x|}|\cos x|}{(1+2x^2)^{5/3}},$$

gdzie M_f jest stałą normującą, przyjmując jako gęstość instrumentalną $g(y|x)$ gęstość rozkładu Cauchy'ego z parametrem skali wynoszącym $\gamma = \frac{1}{\sqrt{|x|}}$, czyli

$$g(y|x) = \frac{\sqrt{|x|}}{\pi} \cdot \frac{1}{1+|x|y^2}$$

Zaproponuj estymator MCMC $\widehat{I}_n^{\mathrm{MCMC}}$ całki I wykorzystujący wygenerowany łańcuch Markowa. Na wykresie zilustruj efektywność estymatora rysując jego asymptotyczne przedziały ufności. Zbadaj (empirycznie) obciążenie estymatora dla różnych czasów burn-in.

2. Zamiast gęstości $g(y|x)$ z podpunktu 1., przyjmij jako gęstość instrumentalną funkcję

$$\tilde{g}(y) = M_{\tilde{g}} \frac{1}{(1+2y^2)^{5/3}},$$

gdzie $M_{\tilde{g}}$ jest stałą normującą. Zmienne z rozkładu o podobnej gęstości (tylko dwuwymiarowego) trzeba było generować w zadaniu 2 z raportu 2 i nie była do tego potrzebna znajomość stałej $M_{\tilde{g}}$.

3. Wykorzystując metodę Rao-Blackwella zaprezentowaną na wykładzie spróbuj zmniejszyć wariancję estymatorów MCMC z poprzenich podpunktów. Porównaj asymptotyczne przedziały ufności poprawionych estymatorów z przedziałami ufności dla pierwotnych.

Rozwiązanie

Zadanie 1.1: Implementacja z gęstością instrumentalną Cauchy'ego

Naszym celem jest oszacowanie wartości całki:

$$I = \int_{-\infty}^{\infty} h(x)dx = \int_{-\infty}^{\infty} \frac{\sqrt{|x|}\cos(x)}{(1+2x^2)^{5/3}}dx$$

Aby to zrobić za pomocą metody Monte Carlo opartej na łańcuchach Markowa (MCMC), musimy przedstawić całkę w postaci wartości oczekiwanej. Przekształcamy całkę w następujący sposób:

$$I = \int_{-\infty}^{\infty} \operatorname{sgn}(\cos(x)) \cdot \frac{\sqrt{|x|}|\cos(x)|}{(1+2x^2)^{5/3}}dx$$

Definiujemy nieznormalizowaną gęstość docelową $f_1(x)$ jako:

$$f_1(x) = \frac{\sqrt{|x|}|\cos(x)|}{(1+2x^2)^{5/3}}$$

Znormalizowana gęstość stacjonarna $f(x)$ ma postać $f(x) = M_f \cdot f_1(x)$, gdzie $M_f = 1/\int_{\mathbb{R}} f_1(x)dx$ jest stałą normalizującą. Całkę I możemy teraz zapisać jako:

$$I = \int_{-\infty}^{\infty} \operatorname{sgn}(\cos(x)) \cdot \frac{f(x)}{M_f}dx = \frac{1}{M_f}\int_{-\infty}^{\infty} \operatorname{sgn}(\cos(x)) \cdot f(x)dx = \frac{1}{M_f} \cdot \mathbb{E}_f[\operatorname{sgn}(\cos(X))]$$

Estymator MCMC dla wartości oczekiwanej $\mathbb{E}_f[\operatorname{sgn}(\cos(X))]$ to średnia ergodyczna:

$$\hat{E}_n = \frac{1}{n}\sum_{i=1}^{n} \operatorname{sgn}(\cos(X_i))$$

gdzie $\{X_i\}$ to próbki z łańcucha Markowa o rozkładzie stacjonarnym $f(x)$. Ostateczny estymator całki I ma postać:

$$\hat{I}_n^{\text{MCMC}} = \frac{1}{M_f} \cdot \frac{1}{n}\sum_{i=1}^{n} \operatorname{sgn}(\cos(X_i))$$

Do generowania próbek $\{X_i\}$ użyjemy algorytmu Metropolisa-Hastingsa. Wymaga on zdefiniowania gęstości instrumentalnej (proposal density). Zgodnie z treścią zadania, w pierwszym podpunkcie jest to gęstość rozkładu Cauchy'ego, zależna od bieżącego stanu łańcucha x_{t-1}:

$$g(y|x) \sim \text{Cauchy}(0, \gamma), \quad \text{gdzie } \gamma = \frac{1}{\sqrt{|x|}}$$

Pełna postać gęstości instrumentalnej to:

$$g(y|x) = \frac{1}{\pi\gamma(1+(y/\gamma)^2)} = \frac{\sqrt{|x|}}{\pi(1+|x|y^2)}$$

W każdej iteracji algorytmu generujemy kandydata $y \sim g(y|x_t)$ i akceptujemy go z prawdopodobieństwem:

$$\alpha(y|x_t) = \min\left(1, \frac{f(y)g(x_t|y)}{f(x_t)g(y|x_t)}\right) = \min\left(1, \frac{M_f \cdot f_1(y)g(x_t|y)}{M_f \cdot f_1(x_t)g(y|x_t)}\right) = \min\left(1, \frac{f_1(y)g(x_t|y)}{f_1(x_t)g(y|x_t)}\right)$$

Jak widać, stała normalizująca M_f skraca się w ilorazie, więc nie musimy jej znać, aby uruchomić algorytm. Niezbędna implementacja znajduje się poniżej.

```r
library(ggplot2)
set.seed(123)

# Funkcja podcałkowa h(x)
h_x <- function(x) {
  (sqrt(abs(x)) * cos(x)) / (1 + 2*x^2)^(5/3)
}

# Gęstość docelowa (nieznormalizowana)
target_f1 <- function(x) {
  return( (sqrt(abs(x)) * abs(cos(x))) / (1 + 2*x^2)^(5/3) )
}

# Gęstość instrumentalna g(y|x)
prop_g_density <- function(y, x_given) {
  gamma <- 1/sqrt(abs(x_given))
  return( dcauchy(y, location = 0, scale = gamma) )
}

# Generowanie z rozkładu instrumentalnego
prop_g_sample <- function(x_given) {
  gamma <- 1/sqrt(abs(x_given))
  return( rcauchy(1, location = 0, scale = gamma) )
}

# Obliczenie teoretycznej wartości całki i stałej normalizującej M_f
integral_f1 <- integrate(target_f1, lower = -Inf, upper = Inf)$value
M_f <- 1 / integral_f1
True_I <- integrate(h_x, lower = -Inf, upper = Inf)$value
```

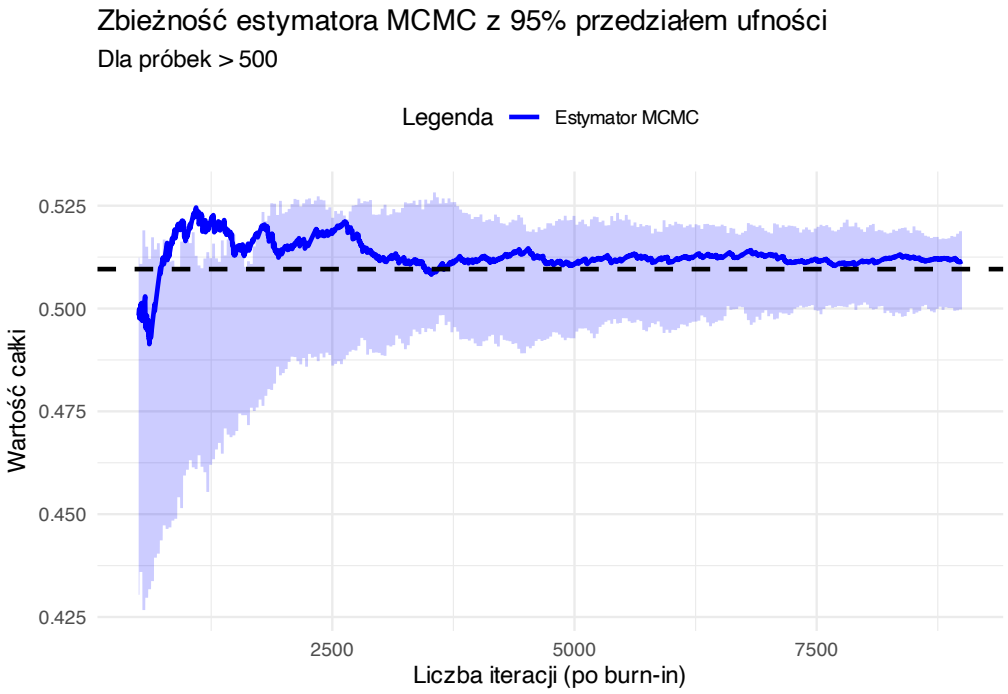
```r
run_metropolis_hastings <- function(n_sim, x0 = 1) {
  chain <- numeric(n_sim)
  chain[1] <- x0

  for (i in 2:n_sim) {
    x_curr <- chain[i-1]

    #  Generowanie propozycji y z g(y|x)
    y_prop <- prop_g_sample(x_curr)

    # Obliczenie stosunku akceptacji
    num <- target_f1(y_prop) * prop_g_density(x_curr, y_prop)
    den <- target_f1(x_curr) * prop_g_density(y_prop, x_curr)

    # Zabezpieczenie przed dzieleniem przez zero
    if (is.na(den) || den == 0) {
      ratio <- 1 # Akceptujemy, jeśli poprzedni stan miał gęstość 0
    } else {
      ratio <- num / den
    }

    alpha <- min(1, ratio)

    # Decyzja o akceptacji
```

```r
    u <- runif(1)
    if (u <= alpha) {
      chain[i] <- y_prop
    } else {
      chain[i] <- x_curr
    }
  }

  return(list(chain = chain))
}
```

```r
set.seed(123)
calc_mcmc_ci <- function(chain, burn_in, M_f, k_blocks = 30) {
  # Usunięcie burn-in
  valid_chain <- chain[(burn_in + 1):length(chain)]
  n <- length(valid_chain)

  # Przekształcenie na wartości, które uśredniamy: sgn(cos(X)) / M_f
  vals <- sign(cos(valid_chain)) / M_f

  # Podział na bloki (Metoda średnich blokowych)
  block_size <- floor(n / k_blocks)

  # Odcinamy końcówkę, żeby bloki były równe
  vals_trimmed <- vals[1:(k_blocks * block_size)]

  # Tworzymy macierz, gdzie każda kolumna to jeden blok
  blocks_matrix <- matrix(vals_trimmed, nrow = block_size, ncol = k_blocks)

  # 4. Obliczenie średnich w blokach (Y_1, ..., Y_k)
  block_means <- colMeans(blocks_matrix)

  # 5. Statystyka na średnich blokowych
  grand_mean <- mean(block_means) # Estymator punktowy

  # Wariancja próbkowa średnich blokowych
  S2_blocks <- var(block_means)

  # Błąd standardowy estymatora
  SE <- sqrt(S2_blocks / k_blocks)

  # 6. Przedział ufności (z CTG dla średnich blokowych)
  alpha <- 0.05
  z_val <- qnorm(1 - alpha/2)

  ci_lower <- grand_mean - z_val * SE
  ci_upper <- grand_mean + z_val * SE

  return(data.frame(
    Estimate = grand_mean,
    SE = SE,
    CI_Lower = ci_lower,
    CI_Upper = ci_upper
  ))
```

```
}
```

```r
set.seed(123)
N <- 10000
burn_in_vis <- 1000
sample_threshold <- 500 # Minimalny rozmiar próbki

# Generowanie łańcucha
res <- run_metropolis_hastings(N, x0 = 1)
chain <- res$chain

# Obliczanie estymat narastająco
valid_chain <- chain[(burn_in_vis+1):N]
vals <- sign(cos(valid_chain)) / M_f
n_seq <- 1:length(vals)

# Średnia skumulowana
cum_means <- cumsum(vals) / n_seq

# Obliczenie CI dla każdego punktu w czasie (metoda średnich blokowych)
ci_results <- t(sapply(n_seq, function(i) {
  res <- calc_mcmc_ci(chain[1:i], burn_in = 0, M_f)

  return(c(res$CI_Lower, res$CI_Upper))
}))

# Tworzenie ramki danych do wykresu
plot_data <- data.frame(
  Iteration = n_seq,
  CumulativeMean = cum_means,
  LowerCI = ci_results[, 1],
  UpperCI = ci_results[, 2]
)

# Filtrowanie danych
plot_data_filtered <- plot_data[plot_data$Iteration > sample_threshold, ]

# Wykres
ggplot(plot_data_filtered, aes(x = Iteration)) +
  geom_line(aes(y = CumulativeMean, colour = "Estymator MCMC"), size = 1) +
  geom_ribbon(aes(ymin = LowerCI, ymax = UpperCI), alpha = 0.2, fill = "blue") +
  geom_hline(yintercept = True_I, linetype = "dashed", size = 1) +
  scale_colour_manual(name = "Legenda", values = c("Estymator MCMC" = "blue", "Prawdziwa wartość" = "re
  labs(
    title = "Zbieżność estymatora MCMC z 95% przedziałem ufności",
    subtitle = paste("Dla próbek >", sample_threshold),
    x = "Liczba iteracji (po burn-in)",
    y = "Wartość całki"
  ) +
  theme_minimal() +
  theme(legend.position = "top")
```

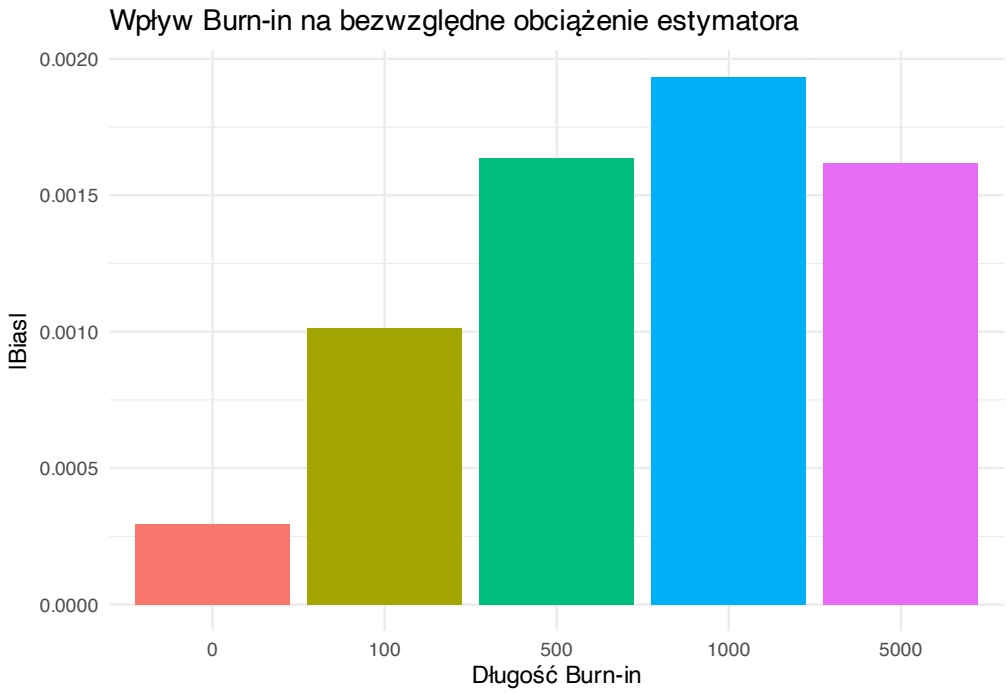


Rysunek 1: Badanie efektywności pierwszego estymatora MCMC

Na rysunku 1 można zaobserwować, jak zachowuję się nasz estymator MCMC w zależności od rozmiaru próbek - zachowuje się dobrze. Oszacowania punktowe nie są znacząco oddalone od teoretycznej wartości całki, a skonstruowane 95% realizację przedziałów ufności zawsze (tj. dla każdego rozmiaru próby) zawierają faktyczną wartość całki. Zbadamy teraz wpływ czasu burn-in na obciążenie estymatora.

```r
set.seed(123)
burn_in_values <- c(0, 100, 500, 1000, 5000)
results <- data.frame()

# Używamy tego samego długiego łańcucha, przycinając go w różnym stopniu
for (b in burn_in_values) {
  if (b < N) {
    est <- calc_mcmc_ci(chain, b, M_f)

    bias <- est$Estimate - True_I

    results <- rbind(results, data.frame(
      BurnIn = as.factor(b),
      Estimate = est$Estimate,
      Bias = bias,
      AbsBias = abs(bias),
      CI_Width = est$CI_Upper - est$CI_Lower
    ))
  }
}

# Wizualizacja Biasu
ggplot(results, aes(x = BurnIn, y = AbsBias, fill = BurnIn)) +
  geom_bar(stat = "identity", show.legend = FALSE) +
```

```r
  labs(
    title = "Wpływ Burn-in na bezwzględne obciążenie estymatora",
    x = "Długość Burn-in",
    y = "|Bias|"
  ) +
  theme_minimal()
```



Rysunek 2: Badanie wpływu czasu burn-in na obciążenie estymatora MCMC

Co ciekawe, wykres 2 pokazuje, że wraz ze wzrostem czasu burn-in, obciążenie naszego wyznaczone estymatora rośnie.

Zadanie 1.2

Postąpimy analogicznie jak w poprzednim podpunkcie, jednak tym razem za gęstość instrumentalną przyjmiemy:

$$\tilde{g}(y|x) = \tilde{g}(y) = M_{\tilde{g}} \frac{1}{(1+2y^2)^{5/3}}$$

, gdzie $M_{\tilde{g}}$ jest stała normującą. Czyli rozważymy szczególny przypadek algorytmu Metropolisa-Hastingsa -niezależne proporcje

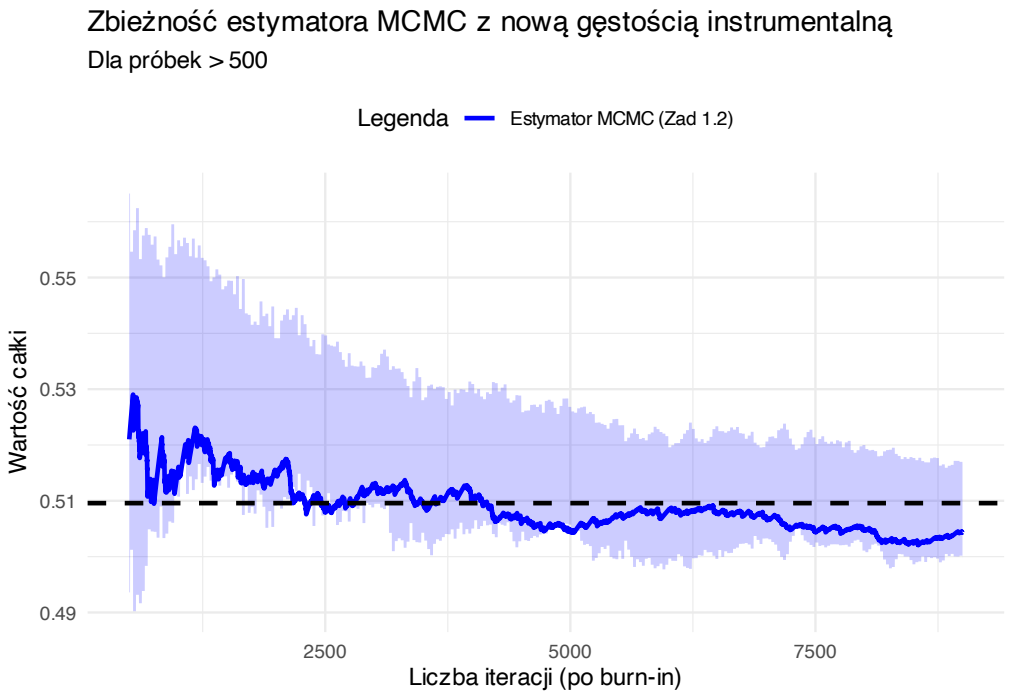
```r
set.seed(123)
# Nowa gęstość instrumentalna g_tilde(y)
# Jest to jądro rozkładu t-Studenta.
# g_tilde(y) propto 1 / (1 + 2*y^2)^(4/3)
# Odpowiada to rozkładowi t-Studenta z nu = 5/3 i skalą sigma = sqrt(3/10)
g_tilde_density <- function(y) {
  nu <- 5/3
  sigma <- sqrt(3/10)
  # Używamy dt() z przeskalowaniem, aby uzyskać gęstość dla niestandardowego rozkładu t
  return(dt(y / sigma, df = nu) / sigma)
}

# Funkcja do próbkowania z g_tilde(y)
g_tilde_sample <- function() {
  nu <- 5/3
  sigma <- sqrt(3/10)
  return(rt(1, df = nu) * sigma)
}
```

```r
set.seed(123)
# Algorytm Metropolisa-Hastingsa z niezależną propozycją
run_independent_mh <- function(n_sim, x0 = 1) {
  chain <- numeric(n_sim)
  chain[1] <- x0

  for (i in 2:n_sim) {
    x_curr <- chain[i-1]

    # Generowanie propozycji y z g_tilde(y) (niezależnie od x_curr)
    y_prop <- g_tilde_sample()

    # Obliczenie stosunku akceptacji

    num <- target_f1(y_prop) * g_tilde_density(x_curr)
    den <- target_f1(x_curr) * g_tilde_density(y_prop)

    if (is.na(den) || den == 0) {
      ratio <- 0
    } else {
      ratio <- num / den
    }

    alpha <- min(1, ratio)

    # Decyzja o akceptacji
    u <- runif(1)
```

```r
    if (u <= alpha) {
      chain[i] <- y_prop
    } else {
      chain[i] <- x_curr
    }
  }

  return(list(chain = chain))
}
```

```r
set.seed(123)
N_task2 <- 10000
burn_in_task2 <- 1000
sample_threshold_task2 <- 500

# Generowanie łańcucha
res_task2 <- run_independent_mh(N_task2, x0 = 1)
chain_task2 <- res_task2$chain

# Obliczanie estymat narastająco
valid_chain_task2 <- chain_task2[(burn_in_task2+1):N_task2]
vals_task2 <- sign(cos(valid_chain_task2)) / M_f
n_seq_task2 <- 1:length(vals_task2)

cum_means_task2 <- cumsum(vals_task2) / n_seq_task2

# Obliczenie CI
ci_results_task2 <- t(sapply(n_seq_task2, function(i) {
  if (i < 30) return(c(NA, NA))
  res <- try(calc_mcmc_ci(chain_task2[1:i], burn_in = 0, M_f), silent = TRUE)
  if (inherits(res, "try-error")) return(c(NA, NA))
  return(c(res$CI_Lower, res$CI_Upper))
}))

# Ramka danych do wykresu
plot_data_task2 <- data.frame(
  Iteration = n_seq_task2,
  CumulativeMean = cum_means_task2,
  LowerCI = ci_results_task2[, 1],
  UpperCI = ci_results_task2[, 2]
)

# Filtrowanie
plot_data_filtered_task2 <- plot_data_task2[plot_data_task2$Iteration > sample_threshold_task2, ]

# Wykres
ggplot(plot_data_filtered_task2, aes(x = Iteration)) +
  geom_line(aes(y = CumulativeMean, colour = "Estymator MCMC (Zad 1.2)"), size = 1) +
  geom_ribbon(aes(ymin = LowerCI, ymax = UpperCI), alpha = 0.2, fill = "blue") +
  geom_hline(yintercept = True_I, linetype = "dashed", size = 1) +
  scale_colour_manual(name = "Legenda", values = c("Estymator MCMC (Zad 1.2)" = "blue", "Prawdziwa wart
  labs(
    title = "Zbieżność estymatora MCMC z nową gęstością instrumentalną",
    subtitle = paste("Dla próbek >", sample_threshold_task2),
```

```
    x = "Liczba iteracji (po burn-in)",
    y = "Wartość całki"
  ) +
  theme_minimal() +
  theme(legend.position = "top")
```



Rysunek 3: Analiza efektywności estymatora MCMC skonstruowanego w oparciu o nową gęstość instrumentalną

Rysunek 3 przedstawia zachowanie "nowego" estymatora MCMC. Jego efektywność jest podobna do efektywności uzyskanej przed pierwszy estymator MCMC.

Zadanie 2: Próbnik Gibbsa (3 pkt)

1. Wykorzystując próbnik Gibbsa napisz program generujący dwuwymiarowy łańcuch Markowa $((X_j, Y_j))_{j\in\mathbb{N}_0}$ o rozkładzie stacjonarnym $\mathcal{N}(0, \Sigma)$, gdzie

$$\Sigma = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}, \quad \rho \in (0, 1).$$

2. Dla jednego ustalonego $\rho \in (0, 1)$ i kilku wybranych punktów początkowych x_0 (bądź y_0) zbadaj średni czas *burn-in*, tzn. czas, po którym prawdopodobieństwa przejścia $\mathbb{P}(X_j \in A|X_{j-1})$ są już wystarczająco bliskie rozkładowi stacjonarnemu.

3. Dla jednego z punktów x_0 z poprzedniego podpunktu, zilustruj otrzymany rozkład docelowy i zbdaj kowariancję zmiennych (X_j, Y_j).

4. Wykorzystaj napisany generator do wyestymowania prawdopodobieństwa

$$\mathbb{P}(e^{Y-X} \geq X), \quad (X, Y) \sim \mathcal{N}(0, \Sigma).$$

Rozwiązanie

1

Aby zastosować próbnik Gibbsa (Gibbs Sampler), musimy znać pełne rozkłady warunkowe dla każdej zmiennej.

Mamy wektor losowy (X, Y) o rozkładzie normalnym $\mathcal{N}(\mathbf{0}, \Sigma)$, gdzie:

$$\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}, \quad \rho \in (0, 1).$$

Z własności wielowymiarowego rozkładu normalnego wiemy, że jeśli (X, Y) mają łączny rozkład normalny, to rozkłady warunkowe $X|Y$ oraz $Y|X$ są również normalne.

Ogólny wzór

Średnia warunkowa: $E[X|Y = y] = \mu_x + \rho \cdot \frac{\sigma_x}{\sigma_y}(y - \mu_y)$

Wariancja warunkowa: $Var(X|Y = y) = \sigma_x^2(1 - \rho^2)$

Obliczenie rozkładów warunkowych

Mamy:

- $\mu_x = 0$, $\mu_y = 0$
- $\sigma_x = 1$, $\sigma_y = 1$ (ponieważ na przekątnej macierzy kowariancji mamy jedynki)
- Korelacja wynosi ρ

Rozkład warunkowy $X|Y = y_j$

$$E[X|Y = y_j] = 0 + \rho \cdot 1(y_j - 1) = \rho y_j$$

$$Var(X|Y = y_j) = 1^2 \cdot (1 - \rho^2) = 1 - \rho^2$$

Zatem

$$X|Y = y_j \sim \mathcal{N}(\rho y_j, 1 - \rho^2)$$

Rozkład warunkowy $Y|X = x_{j+1}$ Ze względu na symetrię macierzy kowariancji i średnich, wyprowadzenie jest identyczne:

$$Y|X = x_j + 1 \sim \mathcal{N}(\rho x_j + 1, 1 - \rho^2)$$

Algorytm Gibbsa:

1. Inicjalizacja: Ustalamy y_0
2. Pętla dla j=1,2,...,N-1:

- losujemy x_{j+1} z rozkładu $\mathcal{N}(\rho y_j, 1 - \rho^2)$
- losujemy y_{j+1} z rozkładu $\mathcal{N}(\rho x_{j+1}, 1 - \rho^2)$
- zapisujemy parę (x_{j+1}, y_{j+1})

poniżej znajduje się kod do powyższego algorytmu

```
gibbs_bvn_systematic <- function(n, rho, x0 = 0, y0 = NULL,
                                 burnin = 0, thin = 1) {
  stopifnot(n >= 1)
  stopifnot(is.numeric(rho), length(rho) == 1, rho > 0, rho < 1)
  stopifnot(burnin >= 0, thin >= 1)

  sigma <- sqrt(1 - rho^2)

  if (is.null(y0)) {
    y0 <- rnorm(1, mean = rho * x0, sd = sigma)
  }

  X <- numeric(n + 1)
  Y <- numeric(n + 1)
  X[1] <- x0
  Y[1] <- y0

  for (j in 1:n) {
    X[j + 1] <- rnorm(1, mean = rho * Y[j], sd = sigma)
    Y[j + 1] <- rnorm(1, mean = rho * X[j + 1], sd = sigma)
  }

  out <- cbind(X = X, Y = Y)

  start_idx <- burnin + 1
  if (start_idx > n + 1) stop("burnin jest za duży względem n.")
  out <- out[start_idx:(n + 1), , drop = FALSE]
  out <- out[seq(1, nrow(out), by = thin), , drop = FALSE]

  return(out)
}
```

2

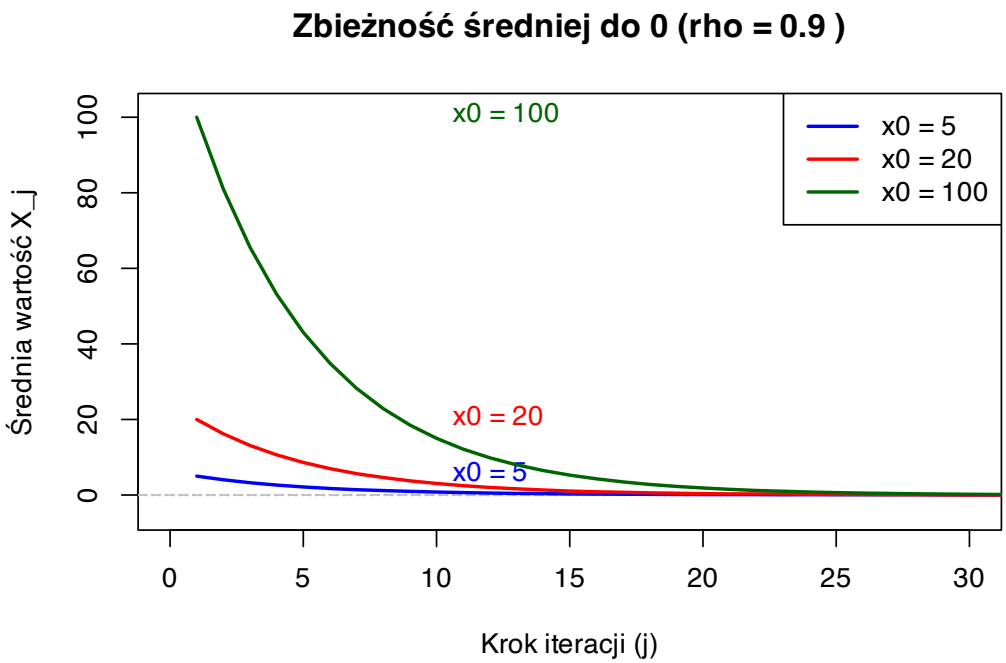
```
rho <- 0.9
start_points <- c(5, 20, 100)
epsilon <- 0.1
n_chains <- 1000
n_steps <- 150
```

```r
calc_theoretical_burnin <- function(x0, rho, eps) {
  return((log(eps) - log(abs(x0))) / (2 * log(rho)))
}

colors <- c("blue", "red", "darkgreen")
plot(NULL, xlim = c(0, 30), ylim = c(-5, 102),
     xlab = "Krok iteracji (j)", ylab = "Średnia wartość X_j",
     main = paste("Zbieżność średniej do 0 (rho =", rho, ")"))
abline(h = 0, lty = 2, col = "gray")
abline(h = c(epsilon, -epsilon), lty = 3, col = "gray")

results <- data.frame(Start=numeric(), BurnIn_Empiryczny=numeric(), BurnIn_Teoretyczny=numeric())

for (k in 1:length(start_points)) {
  x0 <- start_points[k]

  X_paths <- matrix(NA, nrow = n_chains, ncol = n_steps)
  X_paths[, 1] <- x0

  sd_cond <- sqrt(1 - rho^2)

  current_x <- rep(x0, n_chains)

  for (j in 2:n_steps) {
    current_y <- rnorm(n_chains, mean = rho * current_x, sd = sd_cond)

    current_x <- rnorm(n_chains, mean = rho * current_y, sd = sd_cond)

    X_paths[, j] <- current_x
  }

  mean_path <- colMeans(X_paths)

  lines(1:n_steps, mean_path, col = colors[k], lwd = 2)
  text(x = 10, y = x0, labels = paste("x0 =", x0), col = colors[k], pos = 4)

  burn_in_step <- which(abs(mean_path) < epsilon)[1]

  theoretical_step <- calc_theoretical_burnin(x0, rho, epsilon)

  results[k, ] <- c(x0, burn_in_step, round(theoretical_step, 2))
}
kable(results,
      digits = 2,
      col.names = c("Punkt Startowy (x0)",
                    "Burn-In Empiryczny",
                    "Burn-In Teoretyczny"),
      caption = paste("Wyniki zbieżności łańcucha, próg tolerancji epsilon =", epsilon),
      align = 'c')
```

```r
legend("topright", legend=paste("x0 =", start_points), col=colors, lwd=2)
```

Tabela 1: Wyniki zbieżności łańcucha, próg tolerancji epsilon = 0.1

Punkt Startowy (x0)	Burn-In Empiryczny	Burn-In Teoretyczny
5	23	18.56
20	27	25.14
100	33	32.78



Rysunek 4: Zbieżność średniej do 0 (rho = 0.9

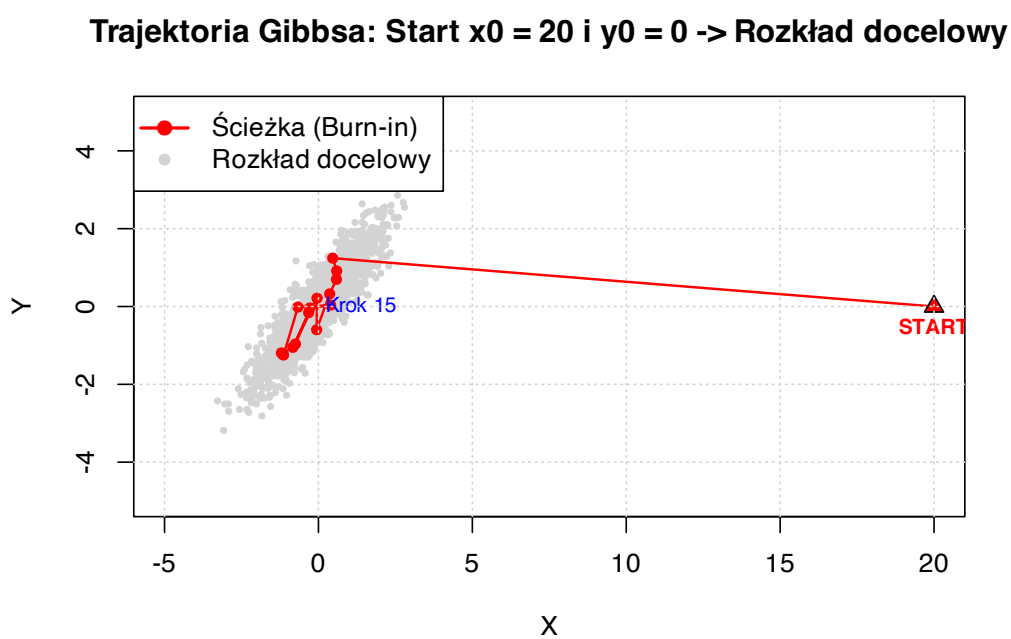
Dla badanego procesu, przy progu ϵ=0.1, okres burn-in rzędu 35-40 iteracji wydaje się być bezpieczną granicą dla dowolnego z rozpatrywanych punktów startowych. Pozwala to na skuteczne wyeliminowanie błędu wynikającego z nielosowego wyboru stanu początkowego.

3

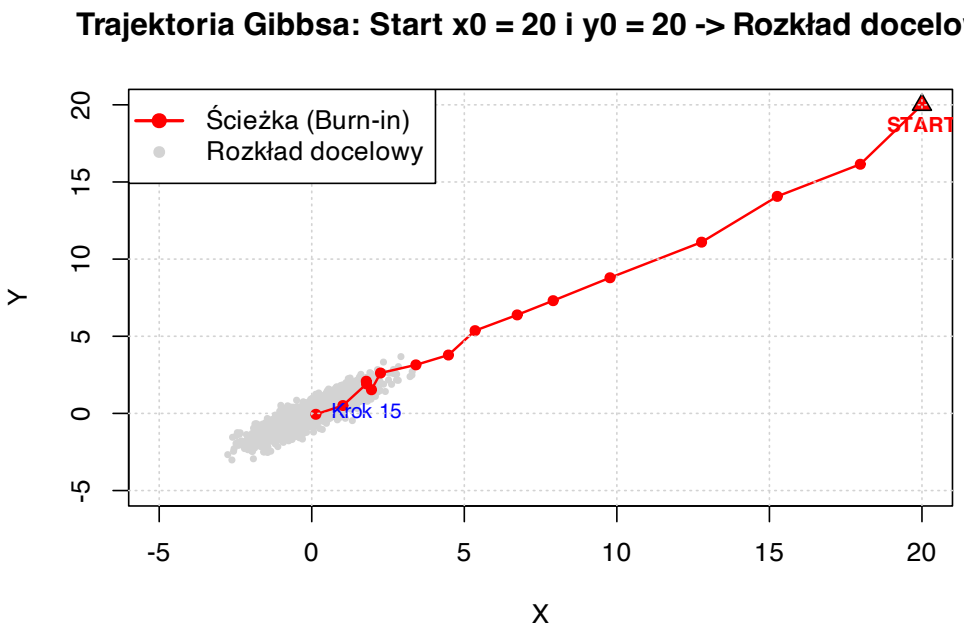
Najpierw zbadajmy jak będzie wyglądać pierwsze 15 kroków w zależności od x0 i y0.

```
rho <- 0.9
x0_val <- 20
n_path <- 15
n_cloud <- 2000

full_chain <- gibbs_bvn_systematic(n = n_path + n_cloud,
                                   rho = rho,
                                   x0 = x0_val,
                                   y0 = 0,
                                   burnin = 0,
                                   thin = 1)

path_data <- full_chain[1:(n_path + 1), ]

cloud_data <- full_chain[(n_path + 2):nrow(full_chain), ]
```

```r
x_limits <- range(c(full_chain[, "X"], -5, 5))
y_limits <- range(c(full_chain[, "Y"], -5, 5))

plot(NULL, xlim = x_limits, ylim = y_limits,
     xlab = "X", ylab = "Y",
     main = paste("Trajektoria Gibbsa: Start x0 =", x0_val, "i y0 = 0 -> Rozkład docelowy"))

points(cloud_data[, "X"], cloud_data[, "Y"],
       col = "lightgray", pch = 16, cex = 0.6)

lines(path_data[, "X"], path_data[, "Y"], col = "red", lwd = 1.5)
points(path_data[, "X"], path_data[, "Y"], col = "red", pch = 19, cex = 0.8)

text(path_data[1, "X"], path_data[1, "Y"], labels = "START",
     pos = 1, col = "red", font = 2, cex = 0.8)
points(path_data[1, "X"], path_data[1, "Y"], col = "black", bg="red", pch = 24, cex = 1.2)

text(path_data[nrow(path_data), "X"], path_data[nrow(path_data), "Y"],
     labels = paste("Krok", n_path), pos = 4, col = "blue", cex = 0.8)
grid()
legend("topleft",
       legend = c("Ścieżka (Burn-in)", "Rozkład docelowy"),
       col = c("red", "lightgray"),
       lty = c(1, NA), pch = c(19, 16), lwd = c(2, NA))
```



Rysunek 5: Trajektoria Gibbsa: Start x0 = 20 i y0 = 0 -> Rozkład docelowy



Rysunek 6: Trajektoria Gibbsa: Start x0 = 20 i y0 = 20 -> Rozkład docelowy

Trajektorie próbkowania Gibbsa pokazują, że niezależnie od punktu startowego, łańcuch szybko zbiega do obszaru wysokiego prawdopodobieństwa rozkładu docelowego, osiągając stabilność przed 15. krokiem.

Tabela 2: Empiryczna macierz kowariancji

	X	Y
X	0.9763	0.7763
Y	0.7763	0.9778

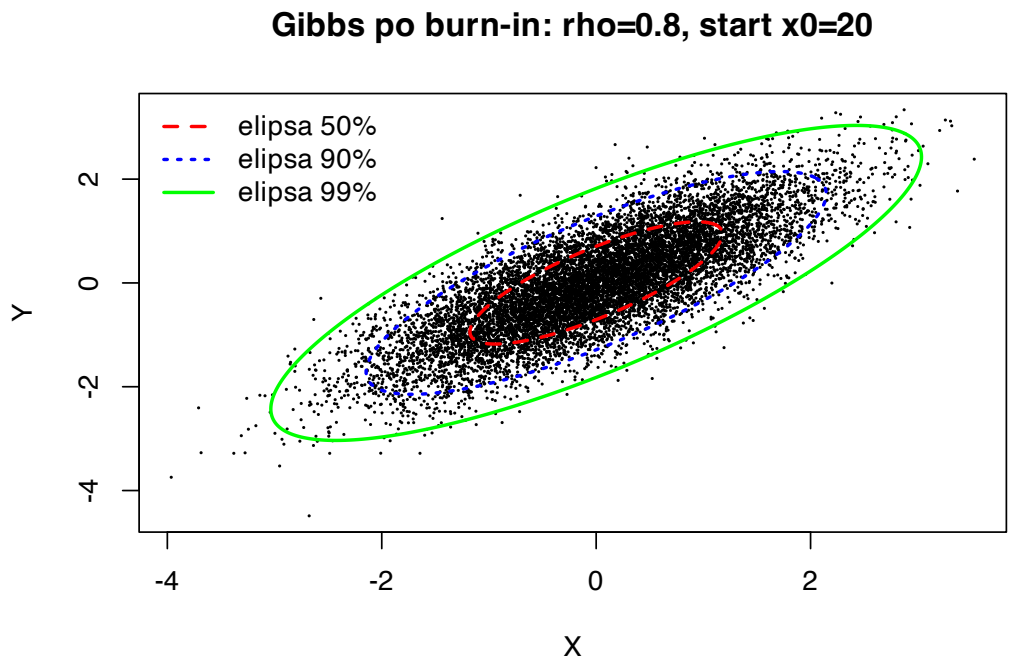
Tabela 3: Teoretyczna macierz kowariancji

	X	Y
X	1.0	0.8
Y	0.8	1.0

```r
set.seed(123)

rho <- 0.8
x0  <- 20
n   <- 60000
burnin <- 5000
thin   <- 5

samp <- gibbs_bvn_systematic(n = n, rho = rho, x0 = x0, burnin = burnin, thin = thin)

Sigma <- matrix(c(1, rho,
                  rho, 1), nrow = 2, byrow = TRUE)
rownames(Sigma) <- c("X", "Y")
colnames(Sigma) <- c("X", "Y")

mu_hat <- colMeans(samp)
S_hat  <- cov(samp)
cor_hat <- cor(samp[, "X"], samp[, "Y"])

kable(S_hat,
      digits = 4,
      caption = "Empiryczna macierz kowariancji",
      align = 'c')
```

```r
kable(Sigma,
      digits = 4,
      caption = "Teoretyczna macierz kowariancji",
      align = 'c')
```

Uzyskana empiryczna macierz kowariancji wykazuje wysoką zgodność z macierzą teoretyczną , co potwierdza, że po okresie burn-in algorytm poprawnie odtwarza strukturę zależności badanego rozkładu dwuwymiarowego.



Rysunek 7: Gibbs po burn-in: rho=0.8, start x0=20

4

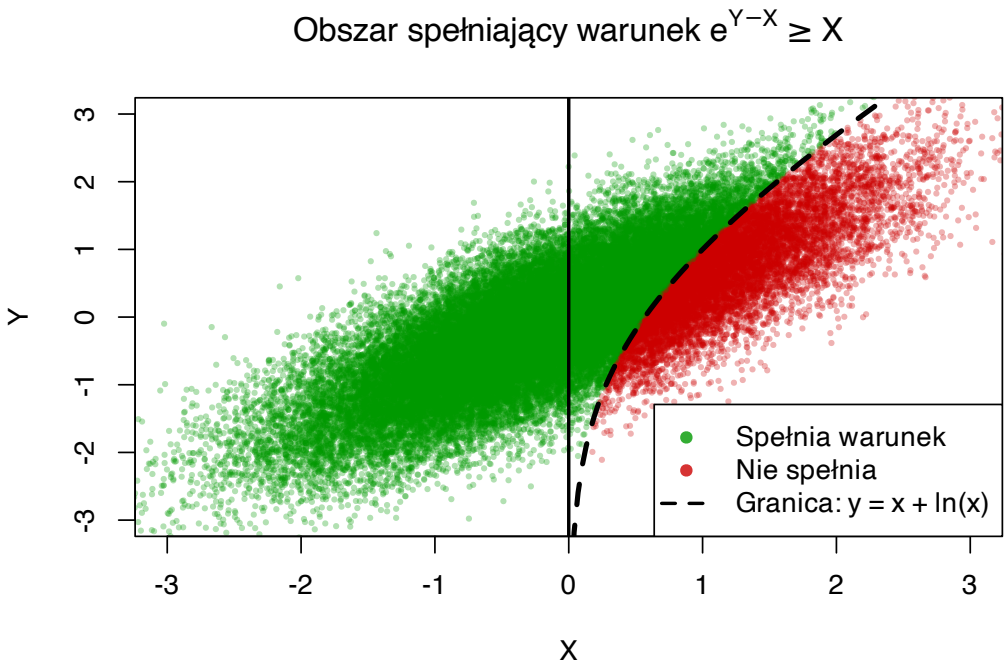
```
set.seed(123)
rho_sim <- 0.8
n_sim   <- 50000
burnin  <- 2000
thin    <- 1

samp <- gibbs_bvn_systematic(n = n_sim, rho = rho_sim, x0 = 0, burnin = burnin, thin = thin)

condition_met <- exp(samp[, "Y"] - samp[, "X"]) >= samp[, "X"]

prob_est <- mean(condition_met)

se_est <- sqrt((prob_est * (1 - prob_est)) / length(condition_met))

integrand <- function(x, r) {
  sd_c <- sqrt(1 - r^2)
  pnorm(x + log(x), mean = r*x, sd = sd_c, lower.tail = FALSE) * dnorm(x)
}

teoria_numeryczna <- 0.5 + integrate(integrand, lower = 0, upper = Inf, r = rho_sim)$value

results_df <- data.frame(
  Metoda = c("Symulacja Monte Carlo", "Teoretyczna (Całka numeryczna)"),
  Prawdopodobieństwo = c(prob_est, teoria_numeryczna),
  Błąd_Std = c(se_est, NA)
)

kable(results_df,
```

Tabela 4: Porównanie wyników prawdopodobieństwa dla rho = 0.8

Metoda	Wynik	Błąd Std. (SE)
Symulacja Monte Carlo	0.8015	0.0018
Teoretyczna (Całka numeryczna)	0.8016	NA

```
    digits = 4,
    caption = paste("Porównanie wyników prawdopodobieństwa dla rho =", rho_sim),
    col.names = c("Metoda", "Wynik", "Błąd Std. (SE)"),
    align = 'c')
```

Estymacja prawdopodobieństwa warunku metodą Monte Carlo (0.8015) okazała się niemal identyczna z wynikiem teoretycznym (0.8016), co przy bardzo niskim błędzie standardowym dowodzi wysokiej precyzji i zbieżności algorytmu



Rysunek 8: Zaznaczenie obszarów spełniających i nie spełniających warunku

Zadanie 3: Estymacja mocy testów (4 pkt)

1. Korzystając z estymacji Monte-Carlo znajdź empiryczne wartości krytyczne statystyk w testach normalności Kołmorowa-Smirnova i Shapiro-Wilka dla poziomów istotności $\alpha_1 = 0.1$ i $\alpha_2 = 0.05$ oraz rozmiarów prób $n_1 = 10$, $n_2 = 100$ i $n_3 = 500$.
2. Oblicz i porównaj empiryczne moce testów normalności Kołmogorowa-Smirnova i Shapiro-Wilka dla poziomów istotności $\alpha_1 = 0.1$ i $\alpha_2 = 0.05$ oraz prób rozmiarów $n_1 = 10$, $n_2 = 100$ i $n_3 = 500$ pochodzących z rozkładu $Beta(2, 2)$.

Rozwiązanie

Zadanie 3.1

Test Kołmogorowa-Smirnova Zastosujemy estymację Monte Carlo w celu oszacowania wartości krytycznej testu normalności Kołmogorowa-Smirnova dla różnych poziomów istotnośći α i dla różnych rozmiarów, n prób (tj. dla $\alpha \in \{0.1, 0.05\}$ oraz $n \in \{10, 100, 500\}$). Najpierw oszacujemy wartość krytyczną dla pierwszego testu. Oszacowane wartości krytyczne przedstawione są w tabeli 5.

```r
set.seed(123)
m<-25000 # Liczba niezależnych próbek do wygenerowania

# Poziomy istotności
alpha1 <- 0.1
alpha2 <- 0.05
alpha_list <- c(alpha1, alpha2)

# Rozmiary prób
n1 <- 10
n2 <- 100
n3 <- 500
n_list <- c(n1, n2, n3)

estimate.ks.p.value <- function(n, alpha = 0.05){
  x<-matrix(rnorm(m*n), nrow=m)
  diff_x<- t(apply(x, 1, function(row) {
    row_s <-sort(row)
    F_emp <- ecdf(row_s)(row_s)
    F0 <- pnorm(row_s)
    c(max(abs(F_emp[1:(n-1)] - F0[1:(n-1)]),abs(F0[2:n] - F_emp[1:(n-1)])),1-F_emp[n])
  }))
  D<-apply(diff_x, 1, max)
  emp_kryt<-quantile(D, probs=1-alpha)

  return(emp_kryt)
}
p.values.ks.results <- matrix(
  nrow = length(alpha_list),
  ncol = length(n_list)
)

rownames(p.values.ks.results) <- paste("alpha =", alpha_list)
colnames(p.values.ks.results) <- paste("n =", n_list)

for (i in 1:length(alpha_list)){
  for (j in 1:length(n_list)){
```

```
    alpha <- alpha_list[i]
    n <- n_list[j]

    p.value <- estimate.ks.p.value(n, alpha)

    p.values.ks.results[i,j] <- round(p.value, 4)
  }
}
```

Tabela 5: Wartości krytyczne testu normalności Kołmogorowa-Smirnova oszacowane metodami Monte Carlo

	n = 10	n = 100	n = 500
alpha = 0.1	0.366	0.121	0.054
alpha = 0.05	0.406	0.134	0.061

Test Shapiro-Wilka Podobnie jak w poprzednim podpunkcie, oszacujemy poziomy krytyczne - tym razem - testu Shapiro-Wilka dokładnie dla tych samych rozmiarów próby i poziomów istotności. Oszacowane p--wartości przedstawione są w tabeli 6.

```
set.seed(123)
# Poziomy istotności
alpha1 <- 0.1
alpha2 <- 0.05
alpha_list <- c(alpha1, alpha2)

# Rozmiary prób
n1 <- 10
n2 <- 100
n3 <- 500
n_list <- c(n1, n2, n3)

estimate.sw.p.value <- function(n, alpha = 0.05){
  # Generowanie macierzy prób
  x <- matrix(rnorm(m*n), nrow=m)

  # Obliczanie statystyki W
  W <- apply(x, 1, function(row) {
    shapiro.test(row)$statistic
  })

  # Wyznaczanie wartości krytycznej.
  emp_kryt <- quantile(W, probs=alpha)

  return(emp_kryt)
}

p.values.sw.results <- matrix(
  nrow = length(alpha_list),
  ncol = length(n_list)
)

rownames(p.values.sw.results) <- paste("alpha =", alpha_list)
```

```r
colnames(p.values.sw.results) <- paste("n =", n_list)

for (i in 1:length(alpha_list)){
  for (j in 1:length(n_list)){
    alpha <- alpha_list[i]
    n <- n_list[j]

    emp_kryt <- estimate.sw.p.value(n, alpha)

    p.values.sw.results[i,j] <- round(emp_kryt, 4)
  }
}
```

Tabela 6: Wartości krytyczne testu normalności Shapiro-Wilka oszacowane metodami Monte Carlo dla różnych poziomów istotności oraz rozmiarów próby

	n = 10	n = 100	n = 500
alpha = 0.1	0.870	0.979	0.995
alpha = 0.05	0.844	0.975	0.994

Zadanie 3.2

Test Kołmogorowa-Smirnova Wykorzystując metody Monte Carlo oszacujemy tym razem moce obu rozważanych do tej pory testów, przyjmując dokładnie te same poziomy istotności oraz rozmiary prób. Próbki zostaną wygenerowane z rozkładu $Beta(2, 2)$. Najpierw oszacujemy moc testu Kolmogorowa-Smirnova. Oszacowane moce przedstawione są w tabeli 7.

```
set.seed(123)
X1<-matrix(rbeta(m*max(n_list),2,2), nrow=m)

estimate.ks.power <- function(n, i,j){
  x1<-X1[, 1:n]

diff_x1<- t(apply(x1, 1, function(row) {
  row_s <-sort(row)
  F_emp <- ecdf(row_s)(row_s)
  F0 <- pnorm(row_s)
  c(max(abs(F_emp[1:(n-1)] - F0[1:(n-1)]),abs(F0[2:n] - F_emp[1:(n-1)])),1-F_emp[n])
}))
  D1<-apply(diff_x1, 1, max)
  emp_kryt <- p.values.ks.results[i,j]

  D1_pow<-sapply(D1,function(x) {x>emp_kryt})
  pow<-mean(D1_pow)

  return(pow)
}

powers.ks.results <- matrix(
  nrow = length(alpha_list),
  ncol = length(n_list)
)

rownames(powers.ks.results) <- paste("alpha =", alpha_list)
colnames(powers.ks.results) <- paste("n =", n_list)

for (i in 1:length(alpha_list)){
  for (j in 1:length(n_list)){
    alpha <- alpha_list[i]
    n <- n_list[j]

    pow <- estimate.ks.power(n,i,j)

    powers.ks.results[i,j] <- round(pow, 4)
  }
}
```

Tabela 7: Moce testu normalności Kolmogorowa-Smirnova oszacowane metodami Monte Carlo dla różnych poziomów istotności oraz rozmiarów próby

	n = 10	n = 100	n = 500
alpha = 0.1	1	1	1
alpha = 0.05	1	1	1

Test Shapiro-Wilka Analogicznie jak dla testu Kolmogorowa-Smirnowa, szacujemy moc testu Shapiro--Wilka, mając na uwadzę, że teraz hipoteza H_0 jest lewostronna. Wyniki szacowania mocy przedstawione są w tabeli 8.

```
set.seed(123)
X1<-matrix(rbeta(m*max(n_list),2,2), nrow=m)

estimate.sw.power <- function(n, i,j){
  x1<-X1[, 1:n]

diff_x1<- t(apply(x1, 1, function(row) {
  row_s <-sort(row)
  F_emp <- ecdf(row_s)(row_s)
  F0 <- pnorm(row_s)
  c(max(abs(F_emp[1:(n-1)] - F0[1:(n-1)]),abs(F0[2:n] - F_emp[1:(n-1)])),1-F_emp[n])
}))
  D1<-apply(diff_x1, 1, max)
  emp_kryt <- p.values.sw.results[i,j]

  D1_pow<-sapply(D1,function(x) {x<emp_kryt})
  pow<-mean(D1_pow)

  return(pow)
}

powers.sw.results <- matrix(
  nrow = length(alpha_list),
  ncol = length(n_list)
)

rownames(powers.sw.results) <- paste("alpha =", alpha_list)
colnames(powers.sw.results) <- paste("n =", n_list)

for (i in 1:length(alpha_list)){
  for (j in 1:length(n_list)){
    alpha <- alpha_list[i]
    n <- n_list[j]

    pow <- estimate.sw.power(n,i,j)

    powers.sw.results[i,j] <- round(pow, 4)
  }
}
```

Tabela 8: Moce testu normalności Shapiro-Wilka oszacowane metodami Monte Carlo dla różnych poziomów istotności oraz rozmiarów próby

	n = 10	n = 100	n = 500
alpha = 0.1	1	1	1
alpha = 0.05	1	1	1

Zadanie 4: Optymalizacja Monte-Carlo (2 pkt)

Korzystając z metody symulowanego wyżarzania znajdź minimum globalne funkcji *stołu Höldera*, czyli funkcji zadanej wzorem:

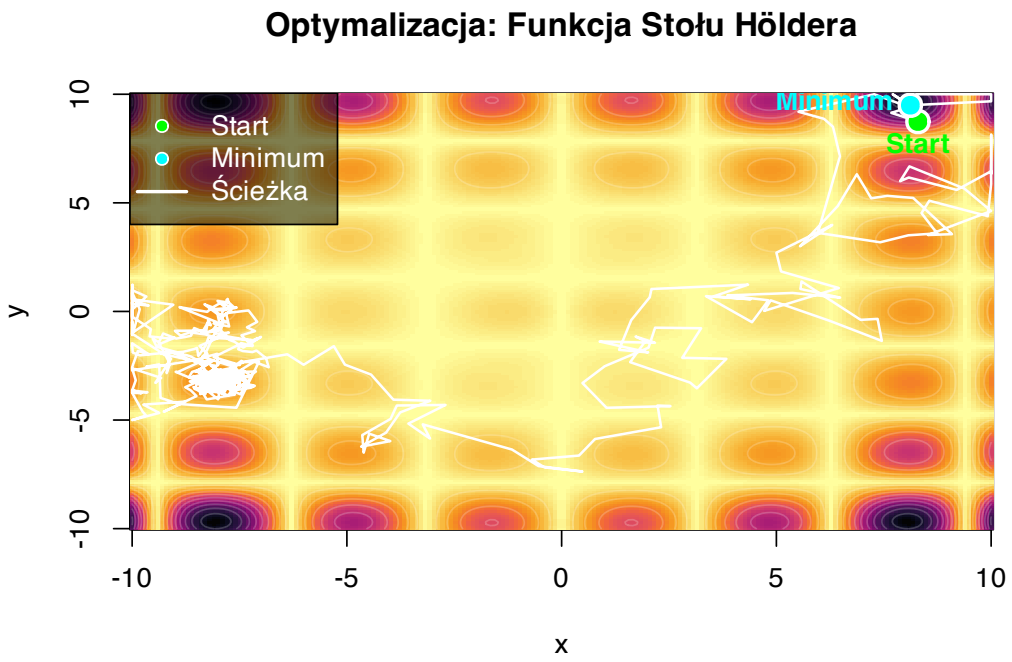
$$f(x,y) = -\left|\sin x \cos y \exp\left(\left|1 - \frac{\sqrt{x^2+y^2}}{\pi}\right|\right)\right| \quad (x,y) \in [-10,10] \times [-10,10].$$

Narysuj wykres funkcji f w postaci mapy cieplnej (heatmap) i zaznacz na nim kolejne punkty wyznaczane w procesie symulowanego wyżarzania.

Rozwiązanie

```r
set.seed(42)

holder_table_func <- function(x) {
  x1 <- x[1]
  x2 <- x[2]

  term1 <- sin(x1) * cos(x2)
  term2 <- exp(abs(1 - sqrt(x1^2 + x2^2) / pi))

  return(-abs(term1 * term2))
}

simulated_annealing <- function(func, lower, upper, max_iter = 10000,
                                T_start = 100, alpha = 0.99) {

  curr_pos <- runif(2, lower, upper)
  curr_val <- func(curr_pos)

  best_pos <- curr_pos
  best_val <- curr_val

  T_curr <- T_start

  path <- matrix(NA, nrow = max_iter, ncol = 2)

  for (i in 1:max_iter) {
    path[i, ] <- curr_pos

    step_size <- 1.0 * (T_curr / T_start) + 0.1
    new_pos <- curr_pos + rnorm(2, mean = 0, sd = step_size)
    new_pos[1] <- max(lower, min(upper, new_pos[1]))
    new_pos[2] <- max(lower, min(upper, new_pos[2]))

    new_val <- func(new_pos)

    delta <- new_val - curr_val

    if (delta < 0 || runif(1) < exp(-delta / T_curr)) {
      curr_pos <- new_pos
      curr_val <- new_val
```

```r
      if (curr_val < best_val) {
        best_pos <- new_pos
        best_val <- new_val
      }
    }

    T_curr <- T_curr * alpha
  }

  return(list(best_pos = best_pos, best_val = best_val, path = path))
}

limits <- 10
result <- simulated_annealing(holder_table_func, -limits, limits,
                              max_iter = 5000, T_start = 10, alpha = 0.995)


grid_seq <- seq(-limits, limits, length.out = 200)
z_vals <- outer(grid_seq, grid_seq, Vectorize(function(x, y) holder_table_func(c(x, y))))

pal <- hcl.colors(100, palette = "Inferno", rev = FALSE)

image(grid_seq, grid_seq, z_vals, col = pal,
      main = "Optymalizacja: Funkcja Stołu Höldera",
      xlab = "x", ylab = "y")

contour(grid_seq, grid_seq, z_vals, add = TRUE, col = rgb(1,1,1,0.2), nlevels = 10, drawlabels = FALSE)

lines(result$path[,1], result$path[,2], col = "white", lwd = 1.5)

points(result$path[1,1], result$path[1,2],
       pch = 21, bg = "green", col = "white", lwd = 2, cex = 1.8)
text(result$path[1,1], result$path[1,2], "Start", pos = 1, col = "green", font = 2)

points(result$best_pos[1], result$best_pos[2],
       pch = 21, bg = "cyan", col = "white", lwd = 2, cex = 1.8)
text(result$best_pos[1], result$best_pos[2], "Minimum", pos = 2, col = "cyan", font = 2)

legend("topleft", legend = c("Start", "Minimum", "Ścieżka"),
       pch = c(21, 21, NA), pt.bg = c("green", "cyan", NA),
       lty = c(NA, NA, 1), col = c("white", "white", "white"), lwd=c(1,1,2),
       bg = rgb(0,0,0,0.5), text.col = "white")
```



Rysunek 9: Optymalizacja: Funkcja Stołu Höldera

Metoda symulowanego wyżarzania skutecznie odnalazła jedno z minimów globalnych funkcji stołu Höldera, co obrazuje biała ścieżka kończąca się w punkcie oznaczonym jako błękitne “Minimum”.

Mapa cieplna pokazuje, że algorytm nie utknął w lokalnych optimach (jasne obszary), lecz dzięki stochastycznemu charakterowi przeszukał znaczną część dziedziny $[-10,10]\times[-10,10]$, ostatecznie zbiegając do obszaru o najniższej wartości funkcji (ciemnofioletowy kolor w rogach).